Less Context, More Accuracy: A Bi-Temporal Memory Engine for LLM Agents Where a Lean Retrieved Context Beats the Full History

Liuyin Wang*
Independent Researcher

June 2026

Abstract

Long-term memory is the missing layer for LLM agents: across sessions they forget, and the common workaround—replaying the entire history into the prompt—is expensive, slow, and, as distractors accumulate, *less* accurate. Most memory systems win on cost or latency but still lose to the full-context baseline on accuracy, and the field's benchmark numbers are reported on inconsistent, non-reproducible harnesses, so the same system appears at wildly different scores across sources. We present ENGRAM, an open-source, dual-process memory engine built on a bi-temporal data model. A fast write path appends lossless episodes without an LLM on the critical path; an asynchronous consolidation path extracts atomic (*subject*, *predicate*, *object*) facts, builds a bi-temporal knowledge graph, and resolves contradictions *without* an LLM call per fact—invalidating, never deleting, so every fact retains provenance and a supersession chain. A hybrid read path fuses dense, lexical, graph, and recency/salience signals, applies a point-in-time ("as-of") temporal filter, and assembles a compact, provenance-tagged context. On the full 500-question LONGMEMEVAL$_S$ benchmark, graded with the *official* category-specific judge prompts, ENGRAM's lean configuration—which answers from a ∼7.9k-token retrieved slice, never the full history—scores **84.4%** versus **78.8%** for the full-context baseline (+5.6 points, McNemar exact $p = 0.009$) while using **10× fewer tokens** (7.9k vs. 79k), with 0 of 500 questions errored under both systems. The advantage grows for a weaker reader: with a small answerer the same slice wins by +12.2 points. On a 200-item slice of LOCOMO, whose ∼16k-token histories leave little for retrieval to remove, the two tie overall while the lean path wins the adversarial and multi-hop categories. We find the gain is load-bearing on the read path being *hybrid*: a facts-only path loses recall, while facts plus retrieved raw chunks recover detail. Beyond the system, we contribute a single neutral, in-repo evaluation harness with the official judge baked in, the full-context baseline in every table, and the raw per-question logs published—and we document the measurement-integrity pitfalls (truncation, home-grown judges, full-history "leaks") that silently distort memory-benchmark numbers. Every number in this paper ships with the command to reproduce it.

1 Introduction

LLM agents are stateless across sessions. The pragmatic fix—concatenate the whole conversation history into the prompt—scales badly on three axes at once: token cost grows linearly with history, latency follows, and accuracy *degrades* as irrelevant turns crowd the window and the model is forced

*Correspondence: liuyinwangthu@gmail.com. Code, raw logs, and the reproducible harness: https://github.com/ly-wang19/engram.

to locate a needle among distractors [12]. A long-term memory layer that stores, structures, and selectively retrieves the past is the natural alternative, and a growing body of systems pursues it [2, 6, 14–16]; see Du [4] for a recent (2026) survey.

Two gaps remain wide open. First, **accuracy**: most memory systems are reported as cheaper or faster than full-context, but *not more accurate*. Beating full-context *on accuracy*—a precisely retrieved slice that outperforms the noisy full window—is the harder and more valuable target, because it turns memory from a cost optimization into a quality improvement. Second, **reproducibility**: memory benchmarks are reported on inconsistent harnesses with different ingestion, answer prompts, and judges, so a single system can appear tens of points apart depending on the source, and different papers give contradictory orderings. In a field where every number is contested, a neutral harness anyone can re-run is itself a contribution.

We address both with ENGRAM, an open-source memory engine, and its in-repo evaluation harness. Our design follows a single principle—*a number we cannot reproduce does not exist*—and a composition thesis: no single mechanism wins, so we compose typed memory, a bi-temporal knowledge graph, multi-signal retrieval fusion, salience decay, and asynchronous consolidation, and build in the seams between them.

Contributions.

1. **A dual-process, bi-temporal memory engine** (§3) with cheap, non-destructive conflict resolution: a contradicted fact is *invalidated* (with a `supersedes` chain and full provenance), never overwritten, and the common case is resolved with *no* LLM call.

2. **The empirical result that a lean retrieved context beats full-context on accuracy** (§5): +5.6 points (84.4% vs. 78.8%) at 10× fewer tokens on the full 500-question LONGMEMEVAL$_S$ under the official judge prompts[1]—i.e. removing distractors *raises* accuracy—and the margin widens to +12.2 for a small answerer (§5.1), together with the finding that a *hybrid* facts-plus-chunks read path is load-bearing (facts alone lose recall).

3. **A neutral, reproducible harness** (§4): one in-repo pipeline with the official judge baked in, the full-context baseline in every table, the same answerer and judge applied to every system by construction, and the raw per-question logs published. We additionally document the measurement-integrity pitfalls we found and fixed.

4. **A per-category and cross-backbone analysis** (§5) isolating where the lean slice wins (multi-session +11.3, single-session-preference +30.0, single-session-user +8.6) and where it loses (knowledge-update −6.4), plus a second benchmark—a labelled 200-item LOCOMO slice (§5.2)—that gives independent evidence for the abstention gate and the multi-hop planner and exposes a date-granularity limitation of fact extraction.

ENGRAM is dual-licensed (AGPL-3.0 plus a commercial license), self-hostable, and runs end-to-end with zero setup—no API keys, no external services—via deterministic offline fallbacks, so the architecture and the demo are reproducible without network access.

[1] The arXiv v1 of this paper reported 83.6% vs. 73.2% (+10.4) at 9.6k vs. 79k tokens on the same 500 questions. That run was graded by a DeepSeek-V3.2 judge served through a third-party endpoint that has since been retired (the model id now returns `NotFound`), so it can no longer be re-run. Every number in this version comes from a fresh full-set run of the current code with the judge moved to DeepSeek's official API (§4). The judge is what moved the absolute scores: the unchanged full-context baseline scored 73.2% and 76.0% in two full-set runs under the retired judge and 77.6%, 77.4% and 78.8% in three runs under the current one, while `engram_lean` moved 83.6% → 84.4%; the gap therefore narrowed from +10.4 to +5.6. The reproduce command in §4 targets the current judge; both logs stay committed.

2 Related Work

Memory systems for LLM agents. MemGPT [14] frames the LLM as an operating system that pages memory in and out of a limited context window. Generative Agents [15] introduce a memory stream with importance, recency, and relevance scoring plus periodic reflection. Mem0 [2] targets production agents with scalable extract-and-store memory. Zep/Graphiti [16] is the closest in spirit to ENGRAM: a temporally-aware knowledge-graph memory with bi-temporal edges. HippoRAG [6] uses a graph plus personalized PageRank for multi-hop retrieval, and its successor HippoRAG 2 [7] reframes the same machinery as non-parametric continual memory. The most recent agentic-memory systems add LLM-driven control over how memories are written and organized: A-MEM [21] builds an interlinked, Zettelkasten-style note network; MemoryOS [9] imposes an operating-system-style short/mid/long-term hierarchy; and GAM [19] decouples memory encoding from consolidation over hierarchical graphs. ENGRAM differs in *composing* a bi-temporal graph with a hybrid (facts + raw chunks) read path and an explicit, cheap conflict-resolution policy, and—distinctively—in shipping the neutral harness that lets these design choices be measured rather than asserted.

Long context and the cost of distractors. "Lost in the middle" [12] shows that LLMs use long contexts unevenly and that accuracy degrades when relevant evidence is buried among distractors. This is the mechanism our headline result exploits: a filtered, precisely retrieved slice can be *more* accurate than the full window, not merely cheaper.

Benchmarks. LONGMEMEVAL [18] evaluates chat assistants on long-term interactive memory across categories (single/multi-session, knowledge-update, temporal reasoning, preference, and abstention), with category-specific judge prompts. LOCOMO [13] evaluates very long-term conversational memory, and more recent benchmarks extend the setting to multimodal histories (Mem-Gallery [1]). We report on LONGMEMEVAL$_S$ with two answerer backbones and on a 200-item LOCOMO slice (§5.2); the full LOCOMO set is in progress (§6).

Retrieval. ENGRAM's read path builds on dense retrieval [10] with modern dense sentence embeddings (the BGE family [20]; we use the English `bge-small-en-v1.5`), classical lexical scoring (BM25 [17]), and rank fusion via Reciprocal Rank Fusion [3], in the retrieval-augmented generation tradition [11]. The dual-process framing draws on the System-1/System-2 distinction in cognitive science [8], and salience decay on the classical forgetting curve [5].

3 The Engram System

ENGRAM is a dual-process memory system: a fast online write path (System-1) and a slow asynchronous consolidation path (System-2) that writes into a typed, bi-temporal memory, which a hybrid read path queries (Figure 1).

3.1 Bi-temporal data model

Two time axes are first-class everywhere, which is what makes knowledge-updates and "as-of" queries intrinsic rather than bolted-on.

- **Episode** — a raw, lossless turn/event, stamped with *event time* (when it happened in the world) and *ingested-at* (transaction time, when we recorded it).

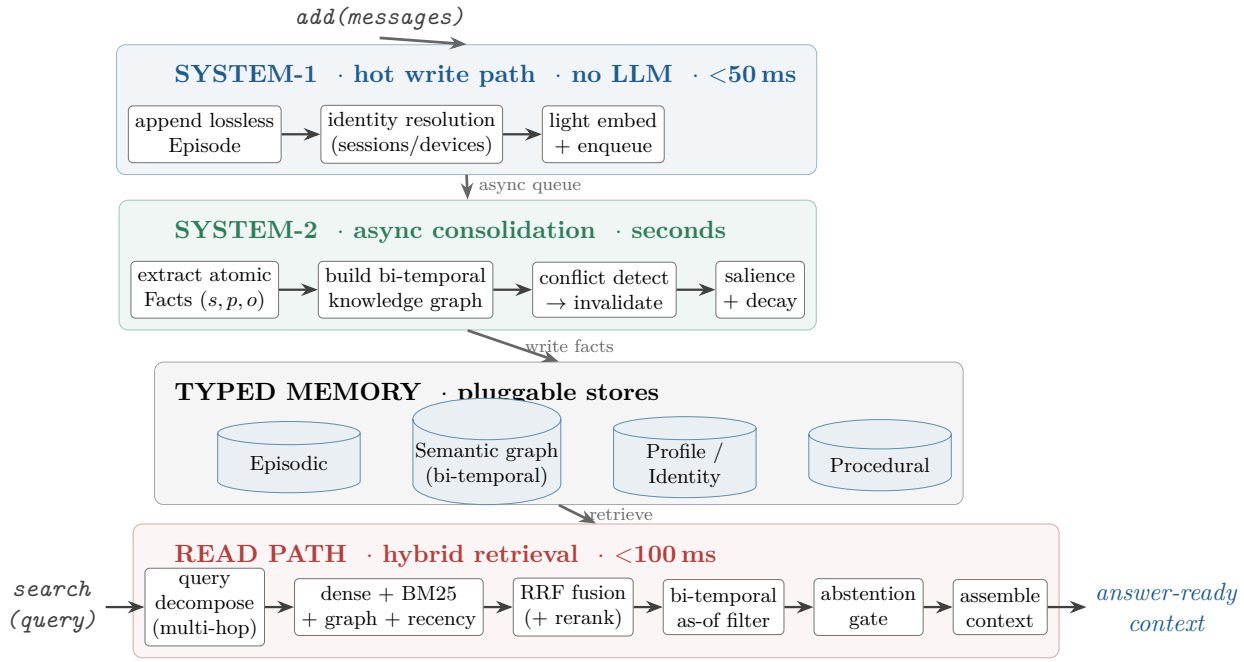

Figure 1: The ENGRAM dual-process architecture. A hot write path (SYSTEM-1) never blocks on an LLM; an asynchronous consolidation path (SYSTEM-2) extracts atomic facts, builds the bi-temporal knowledge graph, and resolves conflicts non-destructively; both feed a typed, bi-temporal memory backed by pluggable stores; and a hybrid read path retrieves a compact, provenance-tagged slice (dense + lexical + graph + recency, fused by RRF), applies an as-of temporal filter and an abstention gate, and assembles the answer context.

- **Fact** — an atomic (*subject*, *predicate*, *object*) claim with surface text, an embedding, a *salience* and *confidence*, and *provenance* (the source episode ids). It carries two time axes: *valid time* (`valid_at` / `invalid_at`, when the claim is true in the world) and *transaction time* (`created_at` / `expired_at`, when we learned or retracted it), plus a `supersedes` pointer to the fact it replaces.
- **Entity / Relation** — graph nodes and edges; edges carry the same bi-temporal stamps.

The invariant: *never hard-delete a contradicted fact—invalidate it* (set `invalid_at`) and keep the history. Every fact can therefore answer "where did this come from?" and "what did it replace?" (Figure 2).

3.2 System-1: the hot write path

On `add(messages)`, ENGRAM appends a lossless episode, resolves identity (linking a user/entity across sessions and devices), computes a light embedding, and enqueues the episode for consolidation. No LLM runs on this path, keeping it within a sub-50ms budget so writes never block the agent.

3.3 System-2: asynchronous consolidation

Off the critical path, the consolidation engine (i) extracts atomic facts from episodes (rule-based by default, LLM-based when a model is configured), (ii) builds the bi-temporal knowledge graph of

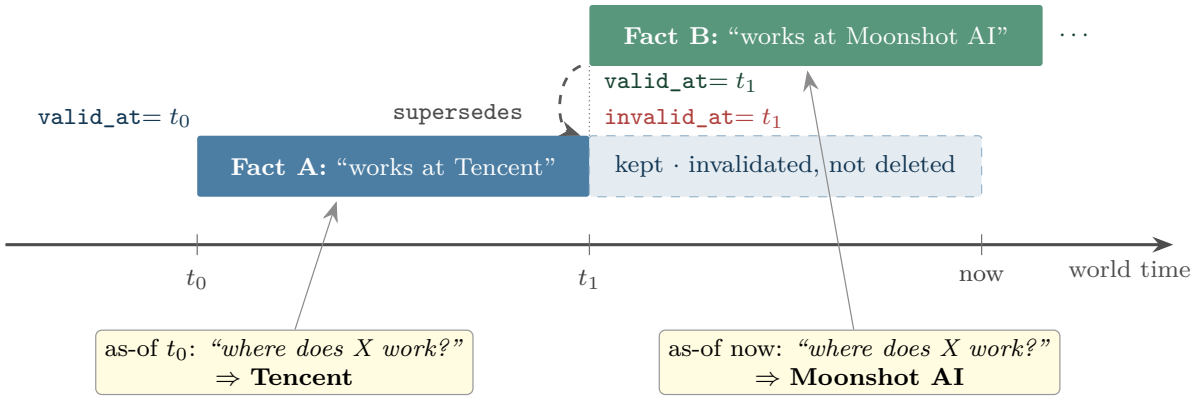

Figure 2: Bi-temporal facts make contradictions and "as-of" queries first-class. When Fact B arrives at t_1, ENGRAM sets A.`invalid_at` $= t_1$ and B.`supersedes` = A: Fact A is *invalidated, not deleted*, so the history (and provenance) survives. A point-in-time query then resolves against the valid fact for that time—TENCENT as-of t_0, MOONSHOT AI as-of now—which is exactly what the knowledge-update and temporal categories require.

entities and relations, (iii) detects conflicts and invalidates superseded facts, and (iv) scores salience and applies decay/reinforcement so unreinforced memories fade and the store stays small and fast. Hierarchical abstraction (session summaries → profile) runs here as well.

3.4 Cheap-then-escalate conflict resolution

When a new fact arrives for an existing (*subject*, *predicate*) slot with a different object, ENGRAM resolves it in increasing order of cost:

1. **Slot match** (exact) signals a likely update; **embedding similarity** catches the same attribute under a different free-form predicate; **content subsumption** (one claim $\subseteq$ the other) separates contradiction from elaboration.
2. If the claims are clearly contradictory and temporally ordered, invalidate the old one (`old.invalid_at` ← `new.valid_at`) and set `new.supersedes` ← `old.id`. *No LLM call.*
3. Only genuinely ambiguous cases escalate to an LLM adjudicator.

This is the cost win over systems that invoke an LLM on every fact, while preserving production-grade temporal correctness and a complete audit trail.

3.5 The hybrid read path

On `search(query)`, ENGRAM (1) understands the query and, for multi-hop questions, decomposes it into sub-queries; (2) retrieves in parallel through four complementary channels—dense semantic, BM25 lexical, graph n-hop from the query's entities, and recency/salience; (3) fuses the ranked lists with Reciprocal Rank Fusion [3] and an optional cross-encoder rerank over the top-k (off by default); (4) applies a bi-temporal "as-of" filter (what we believed was true at time T); (5) passes an abstention gate that declines to answer when the evidence is absent; and (6) assembles a deduplicated, provenance-tagged, token-budgeted context. The per-item score combines all signals:

$$\begin{aligned}\text{score}(item \mid q) = w_{\text{sem}}\cos(q, item) + w_{\text{lex}}\,\text{bm25}(q, item) + w_{\text{graph}}\,\text{prox}(item) \\ + w_{\text{rec}}\,e^{-\Delta t/\tau} + w_{\text{sal}}\,\text{sal}(item),\end{aligned} \tag{1}$$

where the weights are configuration-driven and tuned on the harness rather than hand-set. Crucially, the assembled context is *hybrid*: it contains both the conflict-resolved bi-temporal facts and the most relevant raw session chunks (plus session-level summaries). §5 shows this hybrid composition is necessary—facts alone lose recall.

Pluggable backends. Every external dependency—LLM, embedder, vector store, graph store, lexical index—sits behind an interface with a zero-dependency offline fallback (a hashing embedder, a rule-based extractor, in-memory stores). The end-to-end loop and the unit tests therefore run deterministically with no API keys and no services; real backends (BGE embeddings, LanceDB/Qdrant/pgvector, Kuzu/Neo4j, any LLM via LiteLLM) slot in behind the same interfaces.

4 Experimental Setup

Benchmark. LongMemEval$_S$ [18]: 500 questions, each over a haystack of ∼50 sessions (∼115k tokens), pulled from the public release. Questions span seven categories (Figure 4). We grade with the *official* category-specific judge prompts—including the temporal off-by-one tolerance, knowledge-update old-information tolerance, preference-rubric leniency, the "contains the answer" semantics, and unanswerable (abstention) detection—rather than a home-grown judge.

Models. Embedder: `BAAI/bge-small-en-v1.5` (local, no API key). System-2 fact extractor: `doubao-seed-1.6-flash`. Answerer: `doubao-seed-2.0-pro` for the headline, and the small `doubao-seed-1.6-flash` as a second backbone (§5.1). Judge: DeepSeek's official API (`deepseek-chat`, which served DeepSeek-V4-Flash at the time of the runs), prompted with the official LongMemEval judge prompts; footnote 1 explains why this differs from the arXiv v1 judge. Models are addressed as `provider:model` and resolved via LiteLLM, so any OpenAI-compatible endpoint (OpenAI, DeepSeek, a local model) can be substituted and the same commands run.

Systems. `engram_lean` (our headline) retrieves a small hybrid slice—conflict-resolved bi-temporal facts + the most relevant raw session chunks + session summaries—and answers from *that alone* (∼7.9k tokens), never the full history. `full_context` is the baseline: stuff the entire haystack into the prompt (∼79k tokens). The harness applies the *same answerer and judge to every system*, so any within-run comparison is apples-to-apples by construction.

Retrieval configuration. The hybrid read path fuses five ranked signals by weighted Reciprocal Rank Fusion ($k_{\mathrm{RRF}} = 60$) with the weights in Table 1 (repository defaults, tuned on the harness, not hand-set per question). The lean slice assembles up to 8 conflict-resolved facts, the top-15 fused items, 2 raw session chunks, and 28 session-level summaries; exact flag semantics are in the reproduce command below.

Table 1: Retrieval-fusion weights for Eq. (1) and the RRF constant, as used by the headline `engram_lean` configuration (defaults in `engram/config.py`).

w_{sem}	w_{lex}	w_{graph}	w_{rec}	w_{sal}	k_{RRF}
1.0	0.6	0.8	0.3	0.25	60

Reproduce. The headline run is a single command (raw per-question logs are committed to the repository as `results/run500_v3_main_deepseekjudge.jsonl`):

```
python eval/bench.py -data s -limit 500 \
    -systems engram_lean,full_context \
    -answerer volcano:doubao-seed-2-0-pro-260215 \
    -judge deepseek -extractor volcano:doubao-seed-1-6-flash-250615 \
    -embedder bge-small -reasoning -persona \
    -chunks 2 -topk 15 -extract-k 8 -summ-k 28 -n-summaries 28
```

The second backbone substitutes `-answerer volcano:doubao-seed-1-6-flash-250615`; the LOCOMO slice substitutes `-data locomo -limit 200` (`eval/locomo.py` converts the public LOCOMO release into the harness's item format). `python eval/report.py <run.jsonl>` recomputes every table below from a log, and `python paper/check_numbers.py` asserts that every number in this paper's main text equals a value recomputed from the committed logs.

Measurement-integrity notes. We document the bugs that silently inflate or deflate memory-benchmark numbers, because they are the reason cross-source numbers disagree:

1. **Lean, not full-history (the honest test).** An earlier headline prepended facts *above the entire history*; because that system *contains* full-context, it cannot really lose to it and does not validate the retrieval thesis. The headline is now `engram_lean`, which answers from a ∼7.9k-token retrieved slice.
2. **Full-context truncation.** The baseline was once capped below the haystack size, feeding it only the oldest sessions; this *deflated the baseline*. Fixed so it receives the whole haystack. (Any much lower full-context figure from before this fix is a truncation artifact.)
3. **Official judge, not a home-grown one.** A generic "same info?" judge was *stricter* than the official LongMemEval judge and made scores non-comparable; we use the official prompts verbatim.
4. **Abstention** questions are graded by the official *unanswerable* judge.
5. **Reliability.** The LLM client uses exponential-backoff retry with transient/permanent error classification; the headline run completed with **0 errored questions** of 500 under both systems.
6. **Judge endpoint drift.** The judge behind our arXiv v1 numbers was a third-party endpoint that was retired underneath the documented command, and nothing in the repository noticed until the command failed; the absolute scores moved when we re-ran under a judge we can still reach (footnote 1). A reproducible harness must name its judge beside every number and keep every log it cites committed, which is what we now do.

5 Results

Headline. Table 2 and Figure 3 report the full 500-question result. ENGRAM's lean configuration beats the full-context baseline by +5.6 **points** (84.4% vs. 78.8%) while using **10× fewer tokens** (7.9k vs. 79k), with 0 of 500 errored under both. The filtered slice is not merely cheaper—it is *more accurate* than the full window, consistent with the distractor mechanism of Liu et al. [12]. The margin is statistically significant but not large: across the 500 paired questions `engram_lean` is

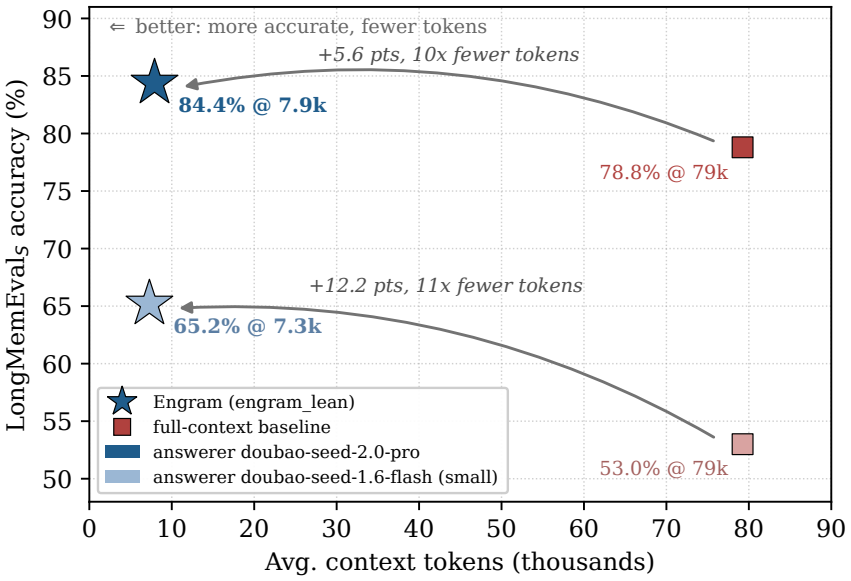

Figure 3: Accuracy vs. average context tokens on LongMemEval$_S$ (500 questions, official judge prompts) for two answerer backbones. For each backbone `engram_lean` (star) sits up and to the left of the full-context baseline (square): +5.6 **points at 10× fewer tokens** for `doubao-seed-2.0-pro`, and +12.2 points at 11× fewer tokens for the small `doubao-seed-1.6-flash` (Table 4)—the weaker the reader, the more it needs the precise slice.

correct on 68 that the baseline misses versus 40 the other way (McNemar's exact test, $p = 0.009$), and a paired bootstrap puts the 95% CI of the gain at +1.6 to +9.6 points. All statistics are recomputed from the committed log by `paper/compute_stats.py`.

Table 2: LongMemEval$_S$, 500 questions, official judge prompts. Same answerer (`doubao-seed-2.0-pro`) and judge (`deepseek-chat`) applied to both systems. Accuracy is shown with a Wilson 95% confidence interval; the paired difference is significant (McNemar exact $p = 0.009$). Latency is end-to-end wall-clock per question including the remote answerer call, p50 / p95.

System	Overall acc. (95% CI)	Avg. context tokens	Latency p50 / p95	Errors
Engram (`engram_lean`)	**84.4%** [81.0, 87.3]	**7.9k**	52.9 s / 118.7 s	0 / 500
full-context baseline	78.8% [75.0, 82.2]	79.2k	14.6 s / 59.2 s	0 / 500

Per-category. Table 3 and Figure 4 break both systems down by category. The lean slice wins four of the six categories and ties one. The margins are largest where the full window has the most to distract from: *single-session-preference* (+30.0; a category that is hard field-wide, and where the baseline reaches only 50.0%), *multi-session* aggregation (+11.3; counting and aggregating across many sessions, where the baseline mostly abstains) and *single-session-user* (+8.6). *Temporal-reasoning* (date-stamped context + as-of filtering) is a modest +2.3. The one loss is *knowledge-update* (−6.4): the baseline sees every version of a changed value in order and applies "most recent wins" itself, whereas the lean slice depends on consolidation having attached the update to the right slot. Of the eight knowledge-update questions the slice loses and the baseline gets, seven end in a

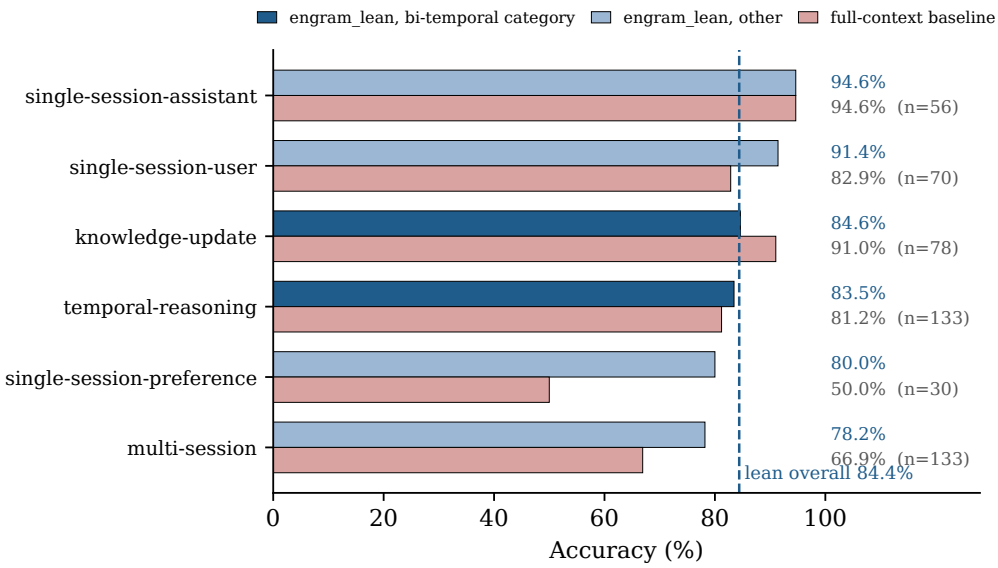

Figure 4: Per-category accuracy of `engram_lean` and the full-context baseline on the full 500-question set (with per-category n; the dashed line is the 84.4% lean overall). The two bi-temporal categories—*knowledge-update* and *temporal-reasoning*—are highlighted: temporal-reasoning is a small win, knowledge-update the one loss. The largest margins are in *single-session-preference* and *multi-session* aggregation.

superseded value or an abstention—consistent with the update not reaching the slice, a consolidation miss rather than a reading error (§6). The 30 *abstention* items, graded by the official *unanswerable* judge and counted inside their host categories above, come out at 90.0% for `engram_lean` vs. 93.3% for the baseline: the abstention gate declines correctly on most of them, at a small cost against a baseline that, as the error analysis below shows, abstains often anyway.

Table 3: Per-category accuracy on LongMemEval$_S$ (500 questions; the run of Table 2). Δ is `engram_lean` minus full-context. The 30 unanswerable (abstention) items are counted inside their host categories; broken out, they score 90.0% (`engram_lean`) vs. 93.3% (full-context).

Category	`engram_lean`	full-context	Δ	n
multi-session	**78.2%**	66.9%	+11.3	133
temporal-reasoning	**83.5%**	81.2%	+2.3	133
knowledge-update	84.6%	**91.0%**	−6.4	78
single-session-user	**91.4%**	82.9%	+8.6	70
single-session-assistant	94.6%	94.6%	+0.0	56
single-session-preference	**80.0%**	50.0%	+30.0	30
overall	**84.4%**	78.8%	+5.6	500

Error analysis: full-context is "lost in the middle." The two systems fail differently. Of the full-context baseline's 106 errors, **57% (60) are abstentions**—it answers "I don't know" even though, by construction, the evidence lies somewhere in its ∼79k-token window; it simply fails to locate it. On the 68 questions where `engram_lean` is correct and full-context is wrong, full-context

abstains on 59% (40) and returns a *wrong* value on the other 28. `engram_lean`, answering from a ∼7.9k-token slice, abstains on 60/500—and 27 of those are among the 30 genuinely unanswerable items. This is the "lost in the middle" effect [12] made quantitative: past a point, more context is *not* more accuracy if the model cannot find the relevant span (counts recomputed from the committed logs by `paper/compute_stats.py`).

The read path must be hybrid. A load-bearing finding from development: a *facts-only* read path—answering purely from extracted (s, p, o) facts—*loses recall* relative to the hybrid path, because extraction drops detail that some questions need verbatim. Adding the most relevant raw session chunks back alongside the conflict-resolved facts restores that detail; the facts contribute conflict-resolved, bi-temporal signal and the chunks restore specificity. The headline configuration is hybrid for exactly this reason, and we caution against shipping facts-only QA. We report this as a design observation; a controlled facts-only ablation on the full 500-question set under the same judge is reported as future work (§6), not claimed here.

Efficiency. The retrieved context averages ∼7.9k tokens, 10× leaner than the ∼79k full-context baseline; the saving is in prompt tokens, the quantity that grows with history. End-to-end latency per question is *higher* for `engram_lean` (p50 52.9 s vs. 14.6 s, Table 2). The harness records wall-clock per question including the remote answerer call and does not instrument retrieval separately, so we report the latency as measured and make no retrieval-latency claim from these logs.

5.1 A second answerer backbone: the weaker the reader, the more it needs the slice

Memory quality should not depend on one model's ability to read our structures, so the same 500 questions were also run with the answerer swapped for the small `doubao-seed-1.6-flash`, everything else unchanged (Table 4). That run was made in June on the revision of that date and graded by the judge that has since been retired (footnote 1), so the table names the code revision and judge per row and we include the June run of the pro answerer as its within-judge sibling; every row is a within-run, same-judge comparison, which is the claim we make. The lean slice wins under every backbone and judge, and the margin is *largest for the weakest reader*: +12.2 points (65.2% vs. 53.0%) for the small answerer, against +3.0 for the pro answerer under the same judge and code, and +5.6 on the current code, at a ∼10–11× token ratio throughout. With the small answerer the lean advantage holds in five of six categories (multi-session +16.5, temporal-reasoning +15.8, single-session-user +12.9, single-session-preference +10.0, knowledge-update +9.0), the exception being single-session-assistant (−1.8), which is near ceiling for both. The reading is simple: a strong model can partly compensate for a noisy window by searching it; a weak one cannot, so removing distractors is worth more to it. That is the regime that matters for cheap and local deployments.

5.2 A second benchmark: a 200-item LOCOMO slice

LOCOMO [13] is a different regime: ten long multi-session conversations whose full text is only ∼16k tokens, a fraction of a LongMemEval$_S$ haystack. We converted its QA pairs into the harness's item format so that the *same* answerer, extractor and judge apply (its adversarial questions map onto the unanswerable judge), and report the **first 200 of its 1,986 items**—a slice, labelled as such; the full run is in progress. Table 5 gives the result. Overall the two systems tie (88.0% vs. 89.0%, −1.0; 0 errors under both) at 3.4k vs. 16.2k tokens (∼5×). That the margin shrinks to zero here is the expected behaviour, not a regression: with a 16k-token history there is little for retrieval

Table 4: Answerer backbones on the same 500 questions, extractor and harness. Gap = `engram_lean` − full-context; token ratio = full-context tokens / lean tokens. Each row is a within-run comparison under one judge; rows differ in code revision and judge as marked, so compare gaps, not levels, across rows. Logs: `results/run500_v3_main_deepseekjudge.jsonl`, `results/headline_500.jsonl`, `results/bb_flash.jsonl`.

Answerer	Code / judge	`engram_lean`	full-context	Gap	Token ratio
`doubao-seed-2.0-pro`	current / `deepseek-chat`	**84.4%**	78.8%	+5.6	10.0×
`doubao-seed-2.0-pro`	June 2026 / retired ARK judge	**79.0%**	76.0%	+3.0	10.9×
`doubao-seed-1.6-flash`	June 2026 / retired ARK judge	**65.2%**	53.0%	+12.2	10.9×

to remove and the full-context reader is not yet lost. The lean advantage is a function of how much distracting history there is—clear on the ∼79k-token LongMemEval windows, nil on 16k—and we expect it to grow, not shrink, with history length.

The per-category split carries the information. Two of Engram's design bets get independent evidence: *adversarial* (the answer is not in the conversation) is 100.0% vs. 91.1% (+8.9, $n = 45$)—the abstention gate declines correctly on every item, where the full-context reader answers some—and *multi-hop* is 78.6% vs. 71.4% (+7.1, $n = 28$), the category the multi-hop query planner targets and one LongMemEval has no separate label for. Single-hop is within noise (−3.6, $n = 83$). The clear loss is *temporal-reasoning* (−16.2, $n = 37$), and its failures are not retrieval misses but *date-granularity* errors: gold answers such as "the week before 9 June 2023" or "the first weekend of August" come back as a single collapsed timestamp, off by weeks or a month, because fact extraction reduces a relative time expression to one point on the axis while the full-context reader still sees the original phrasing. This is a limitation of the current extractor that LongMemEval's more regular date annotations did not expose—the concrete value of a second benchmark—and it is the next target for the bi-temporal model (§6).

Table 5: LOCOMO, first 200 of 1,986 items (a slice; the full run is in progress). Same answerer, extractor and judge as Table 2; log `results/locomo200_main_deepseekjudge.jsonl`, 0 errors.

Category	`engram_lean`	full-context	Δ	n
single-hop	90.4%	**94.0%**	−3.6	83
adversarial	**100.0%**	91.1%	+8.9	45
temporal-reasoning	78.4%	**94.6%**	−16.2	37
multi-hop	**78.6%**	71.4%	+7.1	28
open-domain	**71.4%**	57.1%	+14.3	7
overall	88.0%	**89.0%**	−1.0	200
avg. context tokens	3.4k	16.2k		

6 Discussion and Limitations

We report the result openly rather than as a leaderboard "win," precisely because our thesis is that cross-harness numbers are not comparable. The honest scope of the present evidence:

- **Two backbones from one family; one full benchmark plus a slice.** The headline is LongMemEval$_S$ with a mid-size and a small answerer from the same model family, and the small-answerer run predates the judge migration (§5.1). A frontier-class answerer, a re-run of

the small answerer under the current judge, and the full 1,986-item LOCOMO set (in progress) are the immediate next steps, so that memory quality is shown not to depend on one model's ability to read our structures.

- **Where the slice loses.** Knowledge-update on LongMemEval (−6.4) and temporal-reasoning on LOCOMO (−16.2) are the two categories where the full-context reader beats the lean slice, and both point at consolidation rather than retrieval: an update the extractor misses leaves a stale value in the slice, and a relative time expression collapsed to one timestamp loses the granularity the question needs. Fixing extraction, not adding context, is the lever.
- **Small samples mislead.** During development, small slices repeatedly read far above the full-set truth; we therefore report *only* full-set numbers on LongMemEval, and label the LOCOMO result as a slice.
- **Open categories.** Multi-session aggregation and preference are where headroom remains; the multi-hop query planner and tuned bi-temporal conflict resolution are the levers we expect to move them.
- **Single run per configuration; no controlled component ablation yet.** We report one full-set run per system and backbone and do not quantify run-to-run variance from answerer stochasticity; the paired bootstrap interval (+1.6 to +9.6) is the honest statement of uncertainty for the headline gap, and the full-context baseline's spread across full-set runs under one judge (footnote 1) suggests that noise band is real. Nor do we report a controlled full-set ablation of the read path's components (notably the facts-only vs. hybrid comparison of §5, which we state as an observation). Both are immediate next work.

7 Conclusion

Engram shows that a precisely retrieved, bi-temporal, hybrid context can beat the full-context baseline *on accuracy*—+5.6 points at 10× fewer tokens on the full LongMemEval$_S$ under the official judge prompts, and +12.2 for a small answerer—turning long-term memory from a cost optimization into a quality improvement. Equally, we contribute the neutral, reproducible harness that makes such a claim checkable: the official judge baked in, the full-context baseline in every table, documented measurement-integrity pitfalls, and the raw logs published. In a field where every number is contested, the trustworthy scoreboard is itself the result. The system, the harness, and the logs are open source.

Ethics and Broader Impact

A long-term memory layer necessarily persists personal data across sessions, which raises real privacy obligations. Two of Engram's core design choices are mitigations as much as features: *non-destructive invalidation with full provenance* yields an auditable trail of what was believed, when, and from which source, which makes targeted deletion and "right to be forgotten" requests tractable rather than best-effort; and *salience decay* lets unreinforced personal details fade by default. Because every component has a zero-dependency offline fallback, Engram can run fully locally—no user data need leave the operator's machine—and the AGPL-3.0 license keeps self-hosted data under the operator's control. The principal risks are the obverse of the benefits: a memory store concentrates sensitive information and must be secured, access-controlled, and scoped to genuine

consent; and a memory that confidently surfaces a *stale* or *wrong* fact can mislead, which is precisely why the bi-temporal model and the abstention gate (decline when memory lacks the answer) are first-class rather than optional.

Reproducibility Statement

Every number in this paper is produced by the in-repo harness and backed by committed per-question logs (prediction, gold, correctness, tokens, and latency for every question of every run cited: both answerer backbones on LongMemEval$_S$ and the LOCOMO slice), and `paper/check_numbers.py` asserts that every number in the main text equals a value recomputed from those logs. The end-to-end loop and unit tests run with no API keys and no external services via deterministic offline fallbacks; the benchmark numbers require an OpenAI-compatible model endpoint for the answerer, extractor, and judge, all configurable. The exact command, model identifiers, embedder, and hyper-parameters are in §4; raw logs and the harness are in the repository. The significance tests and confidence intervals in §5 are recomputed from the committed logs by `paper/compute_stats.py`, with no model calls.

References

[1] Yuanchen Bei, Tianxin Wei, Xuying Ning, Yanjun Zhao, Zhining Liu, Xiao Lin, Yada Zhu, Hendrik Hamann, Jingrui He, and Hanghang Tong. Mem-Gallery: Benchmarking multimodal long-term conversational memory for MLLM agents. *arXiv preprint arXiv:2601.03515*, 2026.

[2] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

[3] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.

[4] Pengfei Du. Memory for autonomous LLM agents: Mechanisms, evaluation, and emerging frontiers. *arXiv preprint arXiv:2603.07670*, 2026.

[5] Hermann Ebbinghaus. *Memory: A Contribution to Experimental Psychology.* Teachers College, Columbia University, 1913. Original work published 1885.

[6] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[7] Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. From RAG to memory: Non-parametric continual learning for large language models. In *International Conference on Machine Learning (ICML)*, 2025.

[8] Daniel Kahneman. *Thinking, Fast and Slow.* Farrar, Straus and Giroux, 2011.

[9] Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. Memory OS of AI agent. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025.

[10] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, 2020.

[11] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[12] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics (TACL)*, 12:157–173, 2024.

[13] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[14] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[15] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

[16] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.

[17] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[18] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. *International Conference on Learning Representations (ICLR)*, 2025.

[19] Zhaofen Wu, Hanrong Zhang, Fulin Lin, Wujiang Xu, Xinran Xu, Yankai Chen, Henry Peng Zou, Shaowen Chen, Weizhi Zhang, Xue Liu, Philip S. Yu, and Hongwei Wang. GAM: Hierarchical graph-based agentic memory for LLM agents. *arXiv preprint arXiv:2604.12285*, 2026.

[20] Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. C-Pack: Packed resources for general chinese embeddings. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2024. Introduces the BGE embedding models, incl. `bge-small-en-v1.5`; arXiv:2309.07597.

[21] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025.

A Prompts

For full reproducibility we reproduce the exact prompts the harness uses, verbatim from the repository (non-ASCII characters are normalised for typesetting). The *answerer* system prompt (used with `-reasoning`; it instructs evidence aggregation, most-recent-wins on conflicts, and abstention only as a last resort):

```
You answer the user's question using the provided dated context. The context has two parts: a FACTS
index (a digest) AND the full dated CONVERSATIONS below it. The answer is USUALLY present -- search BOTH
parts thoroughly before concluding anything. The dates are real; use them for temporal reasoning.

DIG FOR THE ANSWER FIRST. Most questions ARE answerable from the history -- your job is to find the
evidence, not to give up. If the FACTS digest doesn't contain it, scan the full CONVERSATIONS below.

WHEN THE QUESTION NEEDS MULTIPLE EVIDENCE PIECES (counting, summing, ordering, date arithmetic, duration,
earliest/latest/most-recent, multi-step lookup, knowledge updated over time):
  EVIDENCE: list EVERY relevant dated item from the context, one per line with its date. Be EXHAUSTIVE
  -- scan the whole history; a missed item makes a count or total wrong. Re-read before finalizing.
  REASON: count distinct items / sum / sort by date / compute the date difference / pick the most recent.
  Show the work in 1-2 lines. For durations compute end_date - start_date explicitly.
  ANSWER: <a single concise final answer -- no reasoning on this line>

FOR DATE/TIME/NUMBER ANSWERS: give the MOST SPECIFIC value the context supports (exact date > month+year
> year; exact duration). Read the exact figure from the text -- don't round (e.g. 27 minutes 45 seconds,
not 28 minutes).

WHEN THE QUESTION ASKS FOR A RECOMMENDATION / PREFERENCE: do NOT refuse, do NOT ask a clarifying question,
do NOT give generic advice. Ground the answer in the user's OWN stated history: name the SPECIFIC people,
places, brands, tools, experiences or constraints they mentioned, and tailor the suggestion to those.

FOR SIMPLE SINGLE-FACT QUESTIONS: go straight to 'ANSWER: <fact>'.

CURRENT-STATE questions ('what is my current/latest X', 'where do I work now'): report ONLY the most
recent value by date and explicitly disregard older, superseded ones. The newest dated statement wins.
When facts conflict, ALWAYS trust the most recent one by date.

ABSTENTION -- a careful LAST resort, only after searching the ENTIRE history (facts AND all conversations)
and the SPECIFIC thing asked is genuinely never stated. Do NOT abstain just because it wasn't in the FACTS
digest. If the question presupposes something the user truly never mentioned, reply exactly 'ANSWER: I
don't know' rather than guessing. Never fabricate a value. The 'ANSWER:' line is REQUIRED.
```

The answer template (the context is the only thing that varies across systems):

```
Today's date: {qdate}

{context}

Question: {question}
Answer:
```

The System-2 fact-extractor system prompt (a non-English worked example is elided for typesetting; values are kept in the user's original language, only the predicate is English snake_case):

```
You are a precise information-extraction engine for a long-term memory system. From a multi-turn
conversation, extract the atomic, durable facts it states about the user and the people/things they
mention (identities, attributes, preferences, relationships, possessions, goals/plans, and events with
their times). Output ONLY a JSON array of objects, each with keys "subject", "predicate", "object",
"text". Use short snake_case predicates (e.g. works_at, lives_in, favorite_color, owns, married_to,
born_in, visited). Capture PREFERENCES explicitly with predicates like likes, dislikes, prefers, avoids,
allergic_to, favorite_<thing>; for each preference or dislike stated, output a SEPARATE fact. Resolve
first-person ('I','my','me') to the user's name when known, otherwise to "user". Capture a stated name as
{"subject":"user","predicate":"name","object":"<Name>"}. LANGUAGE: keep subject and object VALUES (names,
places, brands, free text) in the SAME language the user used -- do NOT translate them; only the predicate
stays English snake_case. "text" is a natural one-sentence statement in the conversation's language. Do
NOT infer or invent facts that are not stated. If there are no durable facts, output [].
```

We grade with the *official* LongMemEval category-specific judge prompts, reproduced verbatim so scores are leaderboard-comparable:

```
[single-session-user / single-session-assistant / multi-session]
I will give you a question, a correct answer, and a response from a model. Please answer yes if the
response contains the correct answer. Otherwise, answer no. If the response is equivalent to the correct
answer or contains all the intermediate steps to get the correct answer, you should also answer yes. If
the response only contains a subset of the information required by the answer, answer no.
Question: {q}   Correct Answer: {a}   Model Response: {r}
Is the model response correct? Answer yes or no only.

[temporal-reasoning]  (as above, plus:)
... do not penalize off-by-one errors for the number of days. If the question asks for the number of
days/weeks/months and the model makes off-by-one errors (e.g., predicting 19 days when the answer is 18),
the model's response is still correct.

[knowledge-update]  (as above, plus:)
... If the response contains some previous information along with an updated answer, the response should
be considered correct as long as the updated answer is the required answer.

[single-session-preference]
I will give you a question, a rubric for desired personalized response, and a response. Answer yes if the
response satisfies the desired response. The model need not reflect all points in the rubric; the response
is correct as long as it recalls and utilizes the user's personal information correctly.

[abstention / unanswerable]
I will give you an unanswerable question, an explanation, and a response. Answer yes if the model correctly
identifies the question as unanswerable (it may say the information is incomplete, or give other
information but not the asked information).
```

B Qualitative Examples

Table 6 shows representative cases drawn from the committed headline log (`results/run500_v3_main_deepseekjud`, where `engram_lean` answers correctly and the full-context baseline does not. Two failure modes recur. **(i) Lost in the middle** [12]: the evidence *is* present in full-context's ∼79k-token window, yet it returns "I don't know," while the lean ∼7.9k-token slice surfaces it. **(ii) Wrong value from a noisy window**: full-context returns a value that is present in the history but is not the one the question asks for (a stale count, the wrong duration, a neighbouring figure) while the lean slice returns the current one. These are illustrative, not cherry-picked headline numbers; all per-question records (prediction, gold, correctness) are in the repository.

Table 6: Representative cases from the LongMemEval$_S$ logs where `engram_lean` is correct and full-context is wrong (ID = the benchmark's question id; predictions verbatim, lightly truncated).

ID	Category	Gold	Engram (`engram_lean`)	full-context
`69fee5aa`	knowledge-update	38	38	*37*
`ba61f0b9`	knowledge-update	6	6	*5*
`0db4c65d`	temporal-reasoning	18 days	18 days	*6 days*
`d851d5ba`	multi-session	$3,750	$3750	*$8750*
`2311e44b`	multi-session	190	190 pages	*I don't know*
`8ebdbe50`	single-session-user	Data Science	Data Science certification	*I don't know*